

Grounding by Falsification: When Compute Is the Wrong Arms Race

Eric Neuman

Abstract. Generative AI reduces the cost of proposing designs, plans, and solutions. It does not automatically reduce the cost of determining which candidates are valid. This asymmetry offers a candidate explanation for AI’s particularly visible impact in software, where compilers, type checkers, test suites, and continuous integration already provide fast, inexpensive ways to reject incorrect candidates. In physical engineering, the decisive evaluation is often fabrication, assembly, laboratory testing, or field deployment.

We introduce **falsification capacity** B_F , the rate at which a workflow’s evaluators dispose of candidates under its sequencing policy, and the *falsification graph*, the dependency structure of those evaluators and the claims they settle. The ratio $\Lambda = G/B_F$ of generation rate to falsification capacity indicates whether a workflow is generation-limited or falsification-limited, and therefore how much leverage it can get from generative AI. For a sequential cycle, Amdahl’s law bounds the speedup from accelerating generation alone at $S_{\max} = 1 + 1/\Lambda_0$, where $\Lambda_0 = t_f/t_g$ is the ratio of falsification time to generation time before acceleration.

Exporting software’s generate-and-test loop to physical design requires sequencing evaluators by expected value, invalidating their verdicts when inputs change, and qualifying evaluators on demonstrated performance rather than authorship. We combine these mechanisms in a *grounded design state*, which jointly encodes candidates, the evaluators that can falsify them, and the provenance and validity of their evidence, so that a change preserves valid evidence, invalidates affected evidence, and recovers rejected candidates. We describe nopekit, an open-source implementation extracted from three earlier projects, including Gripster, a hardware device built with an early manual form of the method, and specify eight experiments. No experimental results are reported here; this paper contributes the framework, the implementation, and the experimental design.

The central claim is conditional. Where $\Lambda > 1$, additional generation grows the evaluation backlog rather than **accepted throughput**, and the same investment directed at faster, cheaper, more independent evaluation returns more; where $\Lambda < 1$, compute remains the right investment. We hypothesize that falling generation costs push more workflows past $\Lambda = 1$, setting off a race to shorten falsification cycle times.

Keywords: generative AI · verification and validation · falsification · generate-and-test · design automation · evaluation bottleneck · engineering design · incremental evaluation

1 Introduction

Generative AI reduces the cost of proposing designs, plans, and solutions. It does not automatically reduce the cost of determining which candidates are valid. A model can propose a circuit, a building detail, or a thermal system in minutes; the evidence needed to accept that proposal may take seconds, days, or months.

Software offers a revealing contrast. A proposed change can encounter a compiler, type checker, test suite, static analyzer, and CI pipeline. These mechanisms are imperfect, but many are fast, repeatable, and already embedded in the development process. Physical engineering also has fast evaluators: geometric checks, analytical calculations, design-rule checks, and simulations. But important claims often remain unresolved until a prototype is printed, a board is fabricated and assembled, a system is tested under load, or a product is used in the field. The most consequential evaluator can be five orders of magnitude slower than the generator (Fig. 2).

Measured productivity effects of generative AI vary widely, from large gains in writing and customer support [10, 42] to a slowdown for experienced open-source developers in one randomized trial [5]. We offer a candidate explanation for part of that variation. The limiting factor is not only how much relevant information exists in a domain or how exposed its tasks

are to language-model assistance [19]. It is also how cheaply and independently a domain can reject bad candidates [53].

We call this capacity **falsification capacity**. By *falsification* we mean evidence, produced by an evaluator, that a specific candidate fails a specific declared claim. The word has two prior owners and we adopt neither wholesale. In philosophy of science it is Popper’s criterion for demarcating empirical theories [45]; our use is narrower, because a scoped verdict about one property of one revision is not a refutation of a theory. In cyber-physical systems testing it denotes the search for an input that violates a temporal-logic specification [2, 21]; our use is broader, because we admit human judgment, physical outcomes, and evaluators with no formal specification. We keep the standard meanings of the neighboring terms [9, 43]: *verification* asks whether an artifact meets its specification, and *validation* whether the specification is the right one.

A collection of evaluators and their dependencies forms a **falsification graph**. It records which claims can be checked, by what means, at what cost and latency, and which results depend on earlier calculations, measurements, or physical artifacts. We say a generative system is **grounded by falsification** when its outputs are coupled to evaluators that can independently reject them. We use *grounding* in the sense of keeping a system in contact with reality, closer to Harnad’s symbol grounding

problem [27] than to the current usage in which grounding means tying model outputs to retrieved sources.

The falsification graph resembles the reinforcement-learning environments (gyms) in which models are trained against automated verifiers [24], but the two answer different questions. A gym’s product is a better generator. A falsification graph’s product is a verdict that can be acted on today, about a specific candidate, from an evaluator the generator does not control. So long as some of a model’s candidates fail, every candidate in a high-value domain must still face evaluators outside the model, and checking is paid for on the candidates that pass as well as those that fail. Better models raise the fraction of candidates that pass. They do not remove the need to check them.

Catalini, Hui, and Wu [12] argue at the macroeconomic level that verification, not execution, becomes the binding constraint as the cost of automation falls, and they locate that constraint primarily in human verification bandwidth. We study the same asymmetry inside engineering workflows, where verification is performed largely by automated and physical evaluators spanning many orders of magnitude in cost and latency. Where their model treats the cost of verification as an aggregate quantity, we decompose it into a falsification graph that can be sequenced, invalidated, and audited. Our qualification and independence rules address a risk they identify, substituting machine checks for human ones, by requiring evaluators to demonstrate that they can fail before their verdicts count as evidence. The claim ledger records where judgment must remain with a named human or institution.

Contributions. The mechanisms we combine are individually established: cost-ordered test sequencing, read-set invalidation, and control-based test qualification each have substantial prior literatures (Section 2). Our contribution is to show (i) that a single measurable ratio, Λ , predicts when assembling them is worth the overhead and when additional generation stops paying, and (ii) that the assembly requires one object, the grounded design state, rather than three tools. We also describe an open-source implementation and the hardware project it was extracted from, and specify eight studies with refutation conditions stated in advance (Section 10).

2 Related work

Performance models for an accelerated phase. Our ceiling is Amdahl’s law [1] applied to a design cycle; Gustafson’s reformulation [25], in which added capacity enlarges the problem rather than shortening it, is the main counter-argument (Section 6.1). Queueing theory supplies the throughput view that Λ approximates [29, 32, 36].

Selective evaluation in software. Test prioritization [18], regression test selection [47], build systems [40], and continuous testing at scale [38] decide what to run after a change; we borrow their read-set discipline. Ordering checks by cost per rejection is the classical solution to sequential testing [52] and to ordering expensive predicates in query optimization [28, 48].

Evaluating the evaluator. Whether a passing test implies the property of interest is the test-oracle problem [4]; mutation testing [16, 31] and metamorphic testing [13] are the main partial responses, and computational engineering formalizes

the question as verification and validation [3, 43]. Coincident failure among independently built versions was demonstrated by Knight and Leveson [33] and modeled by Eckhardt and Lee [17] and Littlewood and Miller [37].

Managing a design space. Carrying sets of candidates rather than one point is set-based concurrent engineering [50]; tradespaces [46], design structure matrices [20], and design rationale [34] represent the space, its dependencies, and the reasons for decisions, and truth maintenance [15] tracks which conclusions rest on which assumptions. Running cheap surrogates ahead of expensive ground truth organizes surrogate-based design [23], multi-fidelity methods [44], and cost-aware Bayesian optimization [49], all value-of-information reasoning [30]. Autonomous laboratories [11, 51] and physics-in-the-loop agentic CAD [7] couple generators to physical or simulated evaluators directly.

3 A ratio and a ceiling

Every numerical value in Sections 3 and 6 is illustrative; no quantity in this paper has been measured in a real workflow.

3.1 The ratio of generation to falsification

Let G be the rate at which a workflow can generate materially distinct candidates for a declared decision, where two candidates are materially distinct if they place a different subset of the decision’s claims at risk. A generator can emit near-duplicates without bound; the distinctness criterion keeps G finite and makes it a rate over the same object as falsification. Let the *survival rate* σ be the fraction of evaluated candidates that pass. Let τ_{pass} be the evaluation time a surviving candidate consumes and τ_{rej} the mean time a rejected candidate consumes before the sequencing cascade of Section 7.1 stops it. We define falsification capacity as the *disposal rate*

$$B_F = [\sigma \tau_{\text{pass}} + (1 - \sigma) \tau_{\text{rej}}]^{-1}, \quad (1)$$

in candidates per unit time, and the ratio

$$\Lambda = \frac{G}{B_F}. \quad (2)$$

B_F so defined depends on the evaluator set, the sequencing policy, and σ : Λ is a property of a workflow, not of a domain, and sequencing is part of the architecture rather than an optimization detail. When $\Lambda \ll 1$, the workflow is *generation-limited*: evaluators wait for candidates, and faster generation translates directly into faster iteration. When $\Lambda \gg 1$, it is *falsification-limited*: candidates arrive faster than their claims can be settled.

In effect, Λ measures how much *leverage* a workflow can get from generative AI. It is defined for one declared decision; where we speak of the Λ of a domain or an industry, we mean a distribution of Λ over the decisions it makes, and summarizing that distribution by a single number is an assumption we do not defend here. E0 would have to make the aggregation rule explicit.

Let *accepted throughput* A be the rate at which candidates survive the falsification a decision requires. Then

$$A \leq \sigma \min(G, B_F). \quad (3)$$

Under the settlement reading $B_F = 1/\tau_{\text{pass}}$, the bound would fail whenever rejections are cheaper than acceptances, because an early-exit cascade disposes of rejected candidates faster than it settles survivors; the disposal-rate definition of Eq. 1 makes it hold.

3.2 A sequential cycle model

Consider a sequential design cycle with two phases: generating a candidate, taking time t_g , and evaluating it against the required acceptance conditions, taking time t_f . If a generative system accelerates generation by a factor s while falsification time remains unchanged, the cycle speedup is

$$S(s) = \frac{t_g + t_f}{t_g/s + t_f}. \quad (4)$$

As generation time approaches zero,

$$S_{\max} = \lim_{s \rightarrow \infty} S(s) = 1 + \frac{t_g}{t_f} = 1 + \frac{1}{\Lambda_0}, \quad \Lambda_0 = \frac{t_f}{t_g}. \quad (5)$$

This is Amdahl’s law [1] applied to a design cycle, and it inherits Amdahl’s fixed-work assumption: it bounds iteration speed on a fixed decision, not the total value of cheap generation (Section 6.1). A workflow that already spends as long falsifying as generating ($\Lambda_0 = 1$) can at most double its cycle rate by accelerating generation alone. If generation takes eight hours and the required evaluation one hour ($\Lambda_0 = 1/8$), the ceiling is $9\times$; if generation takes one day and evaluation two weeks ($\Lambda_0 = 14$), it is about $1.07\times$ (Fig. 1).

The latency ratio Λ_0 and the capacity ratio Λ are related but not interchangeable. G and B_F are capacities; t_g and t_f are latencies of one pass through a sequential cycle. They coincide numerically only under a restrictive reading (one server per stage, unit batch, no buffering) in which $G = 1/t_g$, $B_F = 1/t_f$, and $\Lambda = \Lambda_0$. The two formulations agree on the location of the regime boundary at 1. They do not agree on throughput: sequential completion $\sigma/(t_g + t_f)$ falls short of the pipelined bound $\sigma/\max(t_g, t_f)$ by the factor $\max(t_g, t_f)/(t_g + t_f)$, which is worst, exactly one half, at $\Lambda_0 = 1$. The research question is whether generation and falsification can be operationalized consistently enough to characterize real workflows.

3.3 Interpreting the bound

The ceiling is a sequential model and a local diagnostic. t_f is not the slowest available evaluator but the time to settle the claims a particular decision needs, and a concept review, a prototype decision, and a safety release need different evidence. Real systems pipeline work, so throughput is constrained by evaluator capacity, queueing, and contention rather than by the sum of generation and falsification times; this is the regime Λ describes. Two consequences follow. Read as a utilization, $\Lambda > 1$ is not a steady state but an unstable queue: the backlog grows without bound, so in practice generation is throttled and measured Λ is driven toward 1. And under the latency variability visible in Fig. 2, lead time rises steeply well before Λ reaches 1 [29, 32], so the threshold is a soft knee rather than a corner. Where B_F cannot be estimated directly, Little’s law [36] gives

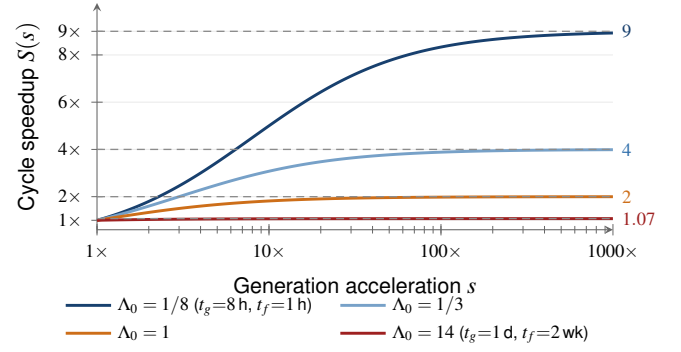


Figure 1. Sequential cycle speedup $S(s)$ from Eq. 4 for several values of $\Lambda_0 = t_f/t_g$, the latency ratio before acceleration (Section 3.2). Dashed lines are the ceilings $S_{\max} = 1 + 1/\Lambda_0$. When falsification dominates the cycle, even unbounded acceleration of generation yields almost no cycle speedup. Illustrative values.

a practical estimator: a team can usually count work-in-process and measure lead time.

Survival rate, independence, and decision scope separate the ceiling from realized gains, and they interact: σ moves with G whenever candidate quality changes; an evaluator that shares the generator’s errors inflates σ without adding evidence; and the decision’s acceptance conditions fix which evaluators count toward t_f at all. The testable consequence is that where generation time falls but required falsification time stays high, accelerating generation alone produces diminishing gains in accepted throughput, and where falsification is cheap and reliable, it translates directly into faster iteration.

4 The falsification graph

Falsification capacity is not a single number. A domain has many evaluators that differ in what they establish and what they cost. We represent the falsification graph as a typed directed graph $\mathcal{F} = (V, E)$ with node set $V = P \sqcup C \sqcup Ev \sqcup Ar \sqcup D$ (parameters, claims, evaluators, evidence artifacts, decisions) and three edge relations: $reads \subseteq Ev \times (P \sqcup Ar)$, $settles \subseteq Ev \times C$, and $requires \subseteq D \times C$. A verdict goes stale when a node its evaluator reads changes, so the invalidation guarantee of Section 7.2 is a reachability property of $\mathcal{F}$ (Fig. 3); $\mathcal{F}$ is a design structure matrix [20] with extra node types.

Each evaluator is characterized by its **latency** (time to a result), **cost** (money and resources per run), **coverage** (the conditions or failure classes it can detect), and **independence** (how far its failure modes differ from those of the generator and other evaluators; Section 7.3). Provenance belongs to the verdict rather than the evaluator (Section 7.2).

The graph makes a distinction often blurred in generated work: a plausible explanation is not an evaluator. A claim is not checked because a model describes a calculation or names a test. It is checked only when a qualified evaluator has run against registered inputs and produced evidence for that claim. Claim statuses preserve this distinction (Table 1).

“Checked” does not mean “true.” It means that a specific evaluator passed a specific test under recorded conditions. An evaluator may be vacuous, incorrectly implemented, incomplete, or inappropriate to the claim. The last of these is the test-oracle

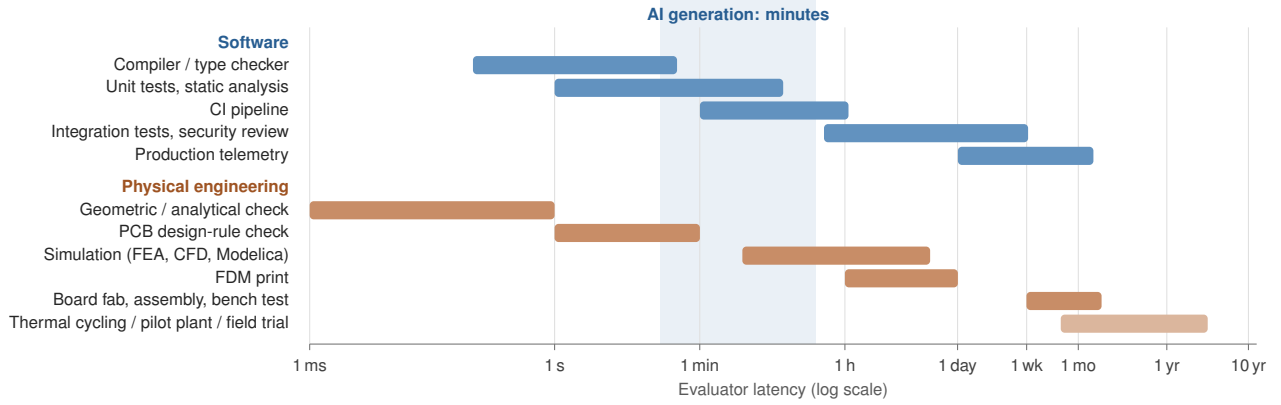


Figure 2. Evaluator latencies span many orders of magnitude, and in physical design the most consequential evaluators sit far to the right of the time a model takes to generate a candidate. Placements are order-of-magnitude estimates from common practice and, for the printing and board bars, from Gripster (Section 9); they are not measurements.

Table 1. Claim statuses. [†]Proposed; not yet implemented in nopekit. Colours are reused in Fig. 3.

Status	Meaning
■ Checked	A qualified evaluator passed for the current registered inputs
■ Settled, non-repeatable[†]	A destructive or one-off evaluation; the verdict stands, the consumed article is recorded
■ Failing	An evaluator rejected the current candidate
■ Stale	Checked evidence no longer matches current inputs, or its control is undemonstrated; graded by refresh cost [†]
■ Assumed	Accepted provisionally with a named reason and owner
■ Pending build	Requires a physical article or other unavailable evaluation
■ Open	An evaluator exists but has not run on the current inputs
■ Gap	No qualified evaluator, or an assumption with no owner
■ Superseded[†]	Replaced by a later verdict on the same claim; kept for audit
■ Skipped/errored	No usable verdict: tool missing, crash, or prerequisite unmet

problem [4]: whether a passing test implies the property of interest has no general solution, and the practical responses, metamorphic relations [13], mutation-based adequacy [31], and validation hierarchies in computational engineering [3, 43], are partial. We address evaluators that cannot fail in Section 7.3 and treat the rest as out of scope.

The falsification graph exposes where a design’s evidence bottoms out. Some claims terminate in closed-form calculations; others in simulations, datasheets, measurements, expert judgments, or physical outcomes. The unresolved portion is not a defect to conceal. It is a map of what remains to be learned.

5 Why physical design changes the problem

Software has mature verification infrastructure, but its evaluators are not uniformly cheap or complete: integration tests, deployment, security review, and production telemetry are slow and costly, which is why test prioritization, regression test selection, and incremental builds exist [18, 38, 40, 47]. The difference in physical workflows is the scale and irreversibility of evaluator costs (Fig. 2). A geometric check takes milliseconds and a print takes hours; a board design-rule check takes seconds and fabrication, assembly, and bench testing take weeks; a simulation takes hours and a pilot plant or field trial takes months or years. When evaluators span that range, deciding what to run, in what order, and after which change becomes part of the

design problem.

The clearest published instance is in materials discovery. Merchant et al. [39] report 2.2 million predicted crystal structures, of which 381,000 are classified as new stable materials; at the time of release, 736 had been independently realized experimentally [14]. An autonomous laboratory operating alongside that work synthesized 41 of 58 targeted compounds over 17 days of continuous operation [51], and a subsequent critique disputed how many of those syntheses established what was claimed [35]. Generation outran physical falsification by about three orders of magnitude in a field with unusually fast falsification infrastructure, and checking the falsifier proved contentious in its own right.

Chip design offers a useful contrast. It shares physical constraints with consumer hardware but relies on electronic design automation (simulation, static timing analysis, formal methods, design-rule and layout-versus-schematic checks) that moves many failure discoveries earlier without eliminating fabrication risk. Consumer hardware has fewer reusable, automated evaluators for system-level behavior, fit, usability, and assembly. Chip design also differs in capital intensity, specialization, market structure, and accumulated investment in tooling, so the contrast motivates a comparison rather than establishing a cause.

Two of the author’s earlier concept studies, which preceded Gripster and nopekit (both at github.com/neuman/stirlingspike), show the mechanism at small scale. In a photovoltaic–thermal model in Modelica, a proposed Stirling stage produced zero modeled power because the vessel never reached the engine’s required temperature difference, and an energy balance gave the project a reason to stop. In a salt-hydrate transport concept, simulation and narrative analysis supported claims labeled “resolved” while degradation over repeated thermal cycles, settled only by a long physical experiment, remained untested. Without a decisive evaluator, fluent analysis makes unresolved assumptions appear settled.

The cross-domain hypothesis is that the leverage a domain gets from generative AI depends not only on task exposure or available knowledge but also on the latency, cost, and independence of the evaluators that convert generated candidates into accepted designs. Testing it requires comparable task-level

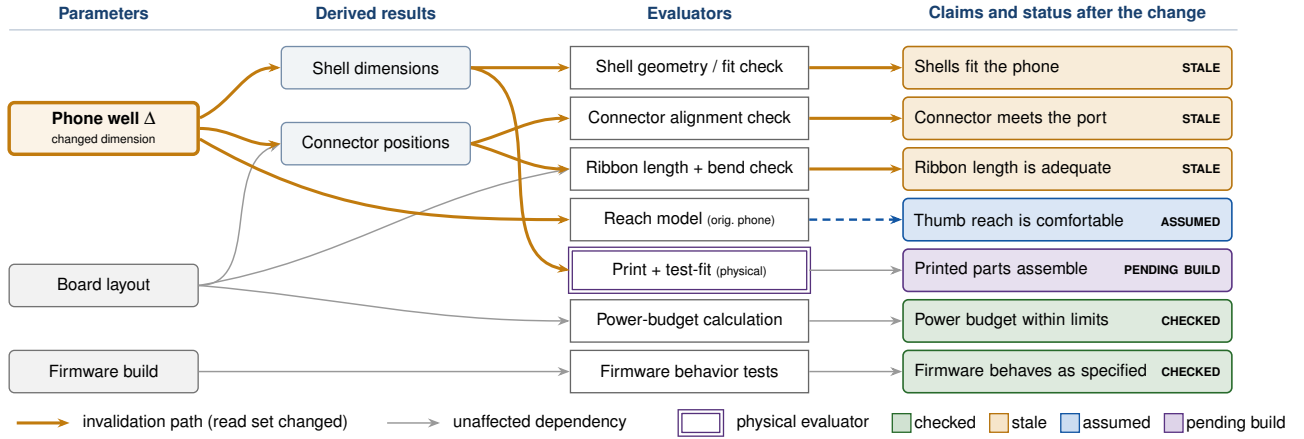


Figure 3. A simplified falsification graph for the Gripster phone-size change (Sections 7.2 and 9). Changing the phone well invalidates every verdict whose registered read set reaches it: shell fit, connector alignment, and ribbon length become *stale*, while power-budget and firmware evidence remain *checked*. Thumb reach reverts to *assumed* because its evaluator was calibrated only for the original phone, and physical fit stays *pending build* until a new article is printed. Node names are illustrative.

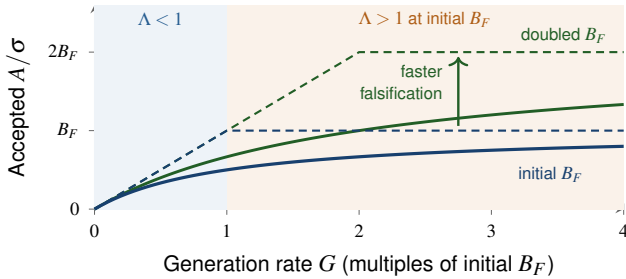


Figure 4. The bound $A \leq \sigma \min(G, B_F)$ of Eq. 3 (dashed) and a smooth saturating alternative $\sigma G B_F / (G + B_F)$ (solid). The corner at $G = B_F$ is an artifact of a deterministic model; under realistic latency variability the transition is a smooth saturation, and lead time rises steeply before Λ reaches 1. Raising B_F lifts both curves; more generation does not.

measures (E0).

6 When compute is the wrong arms race

6.1 The conditional claim

The ratio Λ identifies when additional generation stops paying. In a generation-limited workflow ($\Lambda < 1$), the binding term in Eq. 3 is σG : faster or cheaper generation raises accepted throughput directly, and compute is the right investment. In a falsification-limited workflow ($\Lambda > 1$) the bound becomes

$$A \leq \sigma B_F, \quad (6)$$

in which G does not appear. Holding σ fixed, generating more candidates adds to the backlog without raising accepted throughput, and the marginal return on generation volume approaches zero (Fig. 4). The caveat is not incidental. A generator that produces more candidates by sampling more widely lowers σ , one that produces them by being better raises it, and under the cascade of Section 7.1 a change in σ changes B_F as well. The three quantities are jointly determined, which is why E6 is designed to instrument all three.

The strongest objection is Gustafson’s [25]: cheap generation does not speed iteration on one design; it lets a team carry many candidates into the same evaluation window and keep the best

survivor, raising σ at fixed throughput. That route pays only while the workflow can rank candidates it has not fully settled, and in physical design the cheap evaluators that do the ranking are the ones with the weakest coverage. Beyond that point, more candidates raise the number of plausible survivors, not the number known to work.

Compute spent on generating more candidates is therefore the wrong arms race when $\Lambda > 1$. It remains the right investment when $\Lambda < 1$, and even when $\Lambda > 1$, compute spent on better-targeted candidates raises σ , while compute spent on evaluators, such as validated surrogate simulations, raises B_F . The claim is testable workflow by workflow: for a declared decision with $\Lambda > 1$, it predicts that increasing G leaves A roughly unchanged while increasing B_F or σ raises it, and for $\Lambda < 1$ the reverse. Studies E6 and E7 are designed to test these predictions.

6.2 The cycle-time race

If the claim holds, falling generation costs push more workflows toward $\Lambda > 1$, which raises the value of falsification capacity: AI makes candidates cheap, which makes slow evaluation expensive, which creates incentives to make evaluation faster, which makes more generation useful. Industries then compete not only to build better generators but to make the world capable of rejecting their candidates faster. The movement is already visible in two places. Autonomous laboratories have gone from demonstrations to working instruments within a few years [11, 51], and validated surrogate models, long a specialist technique [23], are becoming a mainstream substitute for expensive simulation [41, 44]. Both are investments in B_F made for reasons the framework predicts. We call the general pattern, which also covers rapid prototyping, hardware-in-the-loop testing, field telemetry, and shorter regulatory loops, the *cycle-time race*.

6.3 What to do instead

For a single workflow, the same logic says what to do instead of buying more generation: measure Λ for the decisions that matter, using Little’s law where B_F cannot be measured directly, and where $\Lambda > 1$, invest in the evaluators that bound

accepted throughput: sequence them by expected cost, invalidate only the evidence that the change has actually expired, qualify model-written evaluators before trusting their verdicts, and retain rejected and unevaluated candidates so that new evaluation capacity can be spent on them. Sections 7 and 8 describe that machinery and the object that holds it.

7 Managing a heterogeneous falsification graph

Exporting software’s generate-and-test loop to physical design requires three capabilities: sequencing, invalidation, and confidence tracking. A claim ledger records their results, and Section 8 combines them into a single object.

7.1 Sequencing evaluators

Let evaluator i have execution cost c_i and probability p_i of rejecting the current candidate. If evaluators are independent, acceptance is conjunctive (every claim must pass), there are no precedence constraints among evaluators, and the objective is to minimize expected cost to the first rejection, then running evaluators in increasing order of

$$\frac{c_i}{p_i} \quad (7)$$

is optimal. This is the classical solution to sequential testing [52], and the same ratio orders expensive predicates in query optimization [28, 48]. Relaxing either structural assumption makes the problem hard: with precedence constraints, and “fabricate before bench-testing” is one, optimal sequencing is NP-hard in general [52]. Batching also breaks the model, since printing six shells costs far less than six single prints, so c_i is a per-candidate marginal cost and batched work calls for a scheduling formulation. We therefore use the ratio as a baseline policy: a bed-fit check precedes a print, and a simulation precedes a board respin (a new fabrication run) when its expected information justifies the cost.

Stopping evaluation leaves an unrun claim open and makes the decision to proceed explicit. Richer policies incorporate false-rejection costs, evaluator dependence, the cost of delaying a physical test, and value of information [30], and the machinery for the central case, cheap surrogates run ahead of expensive ground truth, already exists as multi-fidelity and cost-aware Bayesian optimization [44, 49]. Our contribution is not a new policy but the bookkeeping that makes any such policy applicable to heterogeneous, partly physical evaluators.

Sequencing also undermines itself as generators improve. Ordering changes cost only on candidates that are rejected; a candidate that passes runs every evaluator regardless of order, so expected savings scale with $1 - \sigma$. As $\sigma \rightarrow 1$, the value of Eq. 7 goes to zero, while the value of invalidation and qualification does not.

7.2 Change-driven invalidation

A verdict is meaningful only for the inputs it evaluated. Each evaluator records a read set: the parameters, intermediate results, external files, and evaluator version it consumed. A verdict is then

$$v_i = (o_i, p_i), \quad (8)$$

where o_i is the outcome and p_i a hash of the registered inputs, and the verdict carries its provenance: evaluator version, protocol, and evidence artifacts. When a design changes, the system marks stale the verdicts whose registered read sets changed (Fig. 3). This is the discipline of incremental build systems [40], extended to evidence about physical articles: a firmware change need not invalidate enclosure fit, while a change in device dimensions invalidates shell geometry, connector positions, and reach assumptions.

The guarantee is conditional on complete and correctly enforced read sets, and for physical evaluators that condition cannot be met. A print consumes ambient temperature and humidity, a filament lot, a nozzle’s wear state, and a bed-leveling calibration, none of which is a declared design parameter. We therefore distinguish the *design read set*, which is hashed and gives the invalidation guarantee, from the *environmental read set*, which is declared and recorded and carries an explicit stability assumption rather than a guarantee. A physical verdict is valid under its environmental assumptions; the system makes those assumptions visible and auditable rather than eliminating them. Four refinements follow. For continuous parameters a point hash over-invalidates, so a verdict carries a validity region, the interval over which it is asserted to hold, and goes stale only when the design leaves it. For stochastic evaluators an identical hash does not imply an identical verdict, so a verdict carries a reproducibility status and, for measurements, a margin: 2.01 ± 0.05 against a 2.00 limit is not “checked.” An evaluator that drifts without a version change, such as a printer with slackened belts, carries a recalibration interval whose expiry triggers staleness. And articles from one batch share a batch identity, so one failure can invalidate the batch.

The practical objective is to identify which evidence a change has expired, that is, which verdicts the change has rendered invalid, and re-establish it in a cost-aware order [47].

7.3 Confidence in model-written evaluators

Generative models can write evaluators as well as candidates. This lowers the cost of domain-specific checks but introduces correlated failure risk [33]: the generator can encode the same misunderstanding in the design, the claim, the evaluator, and the test case.

A basic qualification rule is that an evaluator must demonstrate that it can fail: it passes a known-good control and rejects a known-bad control, with both retained and rerunnable. This establishes what we call *discriminative capability*, the ability to distinguish at least one failing input from at least one passing one. It is weaker than verification (the evaluator correctly implements its predicate) and than validation (the predicate supports the claim) [9, 43]; we avoid the term *non-vacuity*, which in formal verification denotes a distinct property of specifications [6]. Discriminative capability is naturally graded: the fraction of mutation-derived faulty inputs an evaluator rejects, its mutation score [16, 31], is more informative than a single control and is what E1 is designed to report.

Independence also needs a number. Let θ be the probability that an evaluator passes a faulty candidate, and θ_0 the same probability for an evaluator written independently of the gen-

erator. The ratio θ/θ_0 is an independence discount, estimable from the mutation-derived controls the qualification rule already collects, and effective falsification capacity is $B_F/(\theta/\theta_0)$. The coincident-failure models of Eckhardt and Lee [17] and Littlewood and Miller [37] give the theoretical basis.

An evaluator’s track record attaches to the evaluator and its operating context, and transfer cannot be assumed: an evaluator that worked for one geometry, material, operating range, or regulatory context can fail under distribution shift. Some claims, including safety certification, legal compliance, containment, and ethics review, require decisions by named human or institutional authorities; the decision to withhold a claim from model-written evaluators is itself recorded in the ledger, with its owner.

7.4 The claim ledger and retained candidates

The ledger maps each claim to its status, evidence, provenance, evaluator, and unresolved assumptions [15]. It also records design decisions and the candidates that were rejected, including why they lost. Rejection types remain distinct (Fig. 5, top band):

- Infeasible:** violates a declared constraint; returns if that constraint changes.
- Dominated:** another feasible candidate is no worse on all declared objectives and better on at least one; returns if the objectives or the competing candidates change.
- Preferred against:** a human or institution chose another candidate under stated preferences; returns if the preference changes.
- Unevaluated:** insufficient evidence exists to compare it; returns when evaluation capacity becomes available.

These are the *return conditions* for each rejection type. Retaining rejected candidates with their return conditions makes the design space revisitable: the ledger records the search history that made the decision intelligible, namely the candidates explored, the evidence for and against each, and the reasons each was rejected. Recovery restores a candidate to consideration; it does not restore its evidence to currency. A candidate rejected as infeasible in 2026 and recovered in 2027 carries verdicts subject to the same staleness rules as any other. Preference records carry an expiry or require re-affirmation on recovery, since a preference about one set of candidates need not transfer to another. And recovery is logical, not economic: after tooling, certification, or a supplier contract is committed, the return condition should state a recovery cost.

8 Grounded design states

The mechanisms of Section 7 need a single object to act on. A conventional generative workflow ships a point: one selected design. In a falsification-limited workflow, that discards the scarcest thing the workflow produced, its evidence, along with every candidate the evidence ruled on and the conditions under which those candidates could return. A generative design system should instead ship the space, not the point. The principle is not new: set-based concurrent engineering, documented at Toyota by Sobek, Ward, and Liker [50], delays convergence

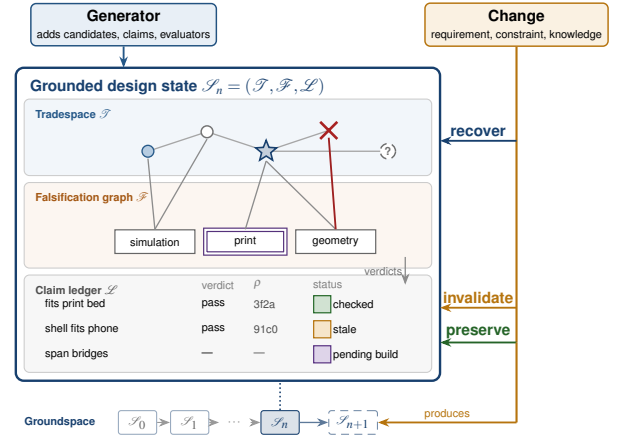


Figure 5. One grounded design state $\mathcal{S}_n = (\mathcal{T}, \mathcal{F}, \mathcal{L})$ within a groundspace, the versioned sequence of states for one design problem (bottom). The generator adds candidates, claims, and evaluators but cannot settle a claim by asserting it. Evaluators in $\mathcal{F}$ rule on the claims of candidates in $\mathcal{T}$, and $\mathcal{L}$ records each verdict with its read-set hash ρ and status. A change preserves verdicts whose read sets are unaffected, invalidates those that are affected, and recovers rejected candidates whose return conditions now hold, producing the next state $\mathcal{S}_{n+1}$. Candidate markers: star, selected; filled circle, preferred against; hollow circle, dominated; cross, infeasible; dashed circle, unevaluated. Status colours follow Table 1.

by carrying sets of alternatives through development. What is new here is that the set arrives with its evidence attached, each surviving candidate carrying the verdicts that kept it alive and each rejected one the condition under which it returns, so the set can be narrowed by a machine rather than only by a review meeting.

Definition. A *grounded design state* is a machine-readable representation of a design problem at one point in its development that jointly encodes a tradespace of candidate configurations and their dependencies, the evaluators applicable to those configurations, the provenance and current validity of the evidence they produce, and the rationale and conditions governing selection, rejection, and reconsideration of candidates. It enables a generator to propose and evaluate design changes while preserving valid evidence, invalidating affected evidence, and recovering previously rejected candidates when requirements, constraints, or available knowledge change.

Formally, a grounded design state is a triple $\mathcal{S} = (\mathcal{T}, \mathcal{F}, \mathcal{L})$ (Fig. 5). The tradespace $\mathcal{T} = (P, V, \phi, u)$ holds the parameters P , the values V of explored candidates and their dependencies, the known feasibility constraints ϕ , and the objective valuation u . The falsification graph $\mathcal{F}$ (Section 4) attaches claims and evaluators to candidates. The claim ledger $\mathcal{L}$ (Section 7.4) records each verdict $v_i = (o_i, \rho_i)$ with its provenance, each rejection with its type, and each rejection’s return condition. Together they represent the explored region of the design space and the evidence boundary accumulated so far. Each point in $\mathcal{T}$ is a *candidate*: one configuration of the design, together with the claims made about it.

Two operations act on $\mathcal{S}$: *propose* : $\mathcal{S} \times \Pi \rightarrow \mathcal{S}$ and *apply* : $\mathcal{S} \times \Delta \rightarrow \mathcal{S}$. *propose* adds parameters, candidates, claims, and

evaluators from a proposal $\pi \in \Pi$, but never a verdict: a claim with no qualified evaluator remains a gap. The generator is allowed to be wrong; it is not allowed to make its wrongness disappear into prose. *apply* takes a change $\delta \in \Delta$ and composes three effects. Writing $\rho(v)$ for the read set whose hash a verdict stores, it *preserves* every verdict with $\rho(v) \cap \delta = \emptyset$, *invalidates* every verdict with $\rho(v) \cap \delta \neq \emptyset$ (Section 7.2), and *recovers* every rejected candidate whose return condition holds in the new state. The invariant is that *propose* is the only operation available to the generator, and its image never intersects the verdict set. Preferences are recorded rather than recomputed: a preferred-against rejection names who supplied the preference, why, and when it should be reconsidered, because preference cannot be derived from engineering state the way feasibility and dominance can.

We call the grounded design states of one design problem its *groundspace*: the versioned sequence $\langle \mathcal{S}_0, \mathcal{S}_1, \dots, \mathcal{S}_n \rangle$ with $\mathcal{S}_{k+1} = \text{apply}(\mathcal{S}_k, \delta_k)$, much as a repository holds a sequence of commits. A grounded design state is the groundspace at one version. The formal groundspace is linear, so the result never depends on how concurrent changes are merged; nopekit stores it in git, where branches can diverge, and merge semantics for conflicting verdicts are not yet defined.

In the terms of Section 6, a grounded design state raises effective falsification capacity without adding evaluators: preserved verdicts are not paid for twice, and recovered candidates arrive with their evidence attached. The size of that gain depends on the change workload, since a project whose every edit touches a root parameter preserves nothing, and it is bought with authoring overhead: read sets must be traced or declared, controls written, and return conditions recorded. E2 and E5 are designed to measure both sides of that trade. nopekit (Section 9) is our implementation.

9 nopekit and Gripster

nopekit (github.com/neuman/nopekit, Apache 2.0), our implementation, is early but working: a Claude Code plugin over a standard-library-only Python spine, organized as claims, gates (its word for evaluators), packs of gates per domain, and a readiness report. Records are files under git, one claim, parameter, or decision per file, and each verdict is cached under the hash of what its gate was seen to read, so a project directory is a groundspace and its checkout a grounded design state. Six packs (beam, solid CAD, FDM printing, fluids, sourcing, thermal) supply 46 gates. The registry refuses a gate without a negative control; read sets are traced rather than declared, with the tracer’s blind spots documented; gates carry cost tiers at which a check can stop; and each parameter records the alternatives that lost to it and why. Ordering by Eq. 7, cross-project confidence, typed return conditions with automatic recovery, and the statuses marked proposed in Table 1 are not yet implemented.

Gripster is a wireless thumb keyboard intended to turn a phone into a two-handed workdeck (Fig. 6a). It was developed across 69 revisions, from one parametric model, toward a fabrication package of two four-layer boards joined by a 20-way FFC ribbon, seven printable shells, two TPU keymats, 78

snap domes on ENIG pads, an nRF52840 module, and ZMK firmware (Fig. 6b; github.com/neuman/gripster). The author built it with a coding agent before nopekit existed, after the two concept studies of Section 5, using an early manual form of the method, and then extracted nopekit from that pipeline; the negative-control requirement exists because a Gripster validator once reported success on physically impossible geometry. Gripster is therefore a history to replay, not a demonstration of nopekit in use. It is one project by the framework’s author, and the method was learned from it, so a replay is in-sample: it supports claims about mechanism, not about benefit or transfer, which need a project nopekit did not learn from (E5).

A phone-size change exercises the invalidation mechanism (Fig. 3). Changing the phone well should stale shell dimensions, connector positions, ribbon length, and fit claims while leaving firmware and electrical calculations untouched, and thumb reach reverts to “assumed” because its evaluator was calibrated for the original phone. The change is in the revision history, and E2 is designed to replay it through nopekit and compare its invalidations with what the change actually required: correct, spurious, and missed, with each miss traced to the read set that omitted it.

A recorded false acceptance. The history also records a ribbon configuration that passed its electrical checks and failed mechanical fit when assembled: a set of checked claims that a physical article contradicted. It tests the distinction between checked and true against an outcome, and it is the first case for E4, to be reported in full: the claim, the evaluator that passed it, its read set and controls, what the article showed, whether the ledger represented the contradiction, and what the read set or coverage would have needed for the framework to catch it. Physical prints also settled claims directly: Fig. 6c shows faceplate versions, each of which consumed a print, and Fig. 6d a shell whose span failed to bridge.

10 Empirical program

No results are reported in this paper. This section specifies the studies that would support, refine, or falsify the framework, the measurements each requires, and the outcomes that would count against it. The questions are whether falsification properties explain cross-domain variation in AI leverage, whether accepted throughput saturates once Λ exceeds 1, and whether managing falsification explicitly improves engineering workflows. Table 2 lists the studies, ordered so that those runnable from existing assets come first: E2 and E3 replay the Gripster history, with E3 estimating rejection probabilities on one portion of it and evaluating policies on held-out changes, and E1 audits the 46 existing evaluators.

E6 — Generation saturation. In a workflow with measured G and B_F , raise the generation rate while holding the falsification graph fixed, and measure accepted throughput, backlog, survival rate, work-in-process, and lead time. Both ways of raising G should be run, since sampling one generator more aggressively is expected to lower σ and substituting a better generator to raise it. Because $\Lambda > 1$ is unstable, the predicted signature is a plateau in A together with linear growth in work-in-process

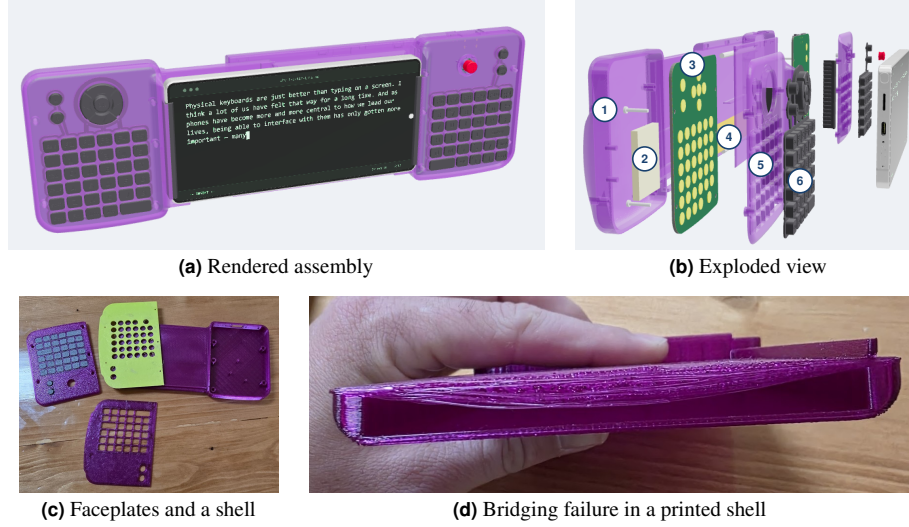


Figure 6. Gripster. (a) A phone docked between the keyboard halves. (b) Exploded view: (1) printed shell, (2) battery, (3) keyboard PCB, (4) FFC ribbon, (5) faceplate, (6) TPU keymat. (c) Faceplate versions and a shell; each print was a physical evaluation. (d) A bridging failure: a shell’s unsupported span sagged into strands.

Table 2. Proposed studies, ordered by what each requires. None has been run.

ID	Question	Primary measures	Requires
E2	Is invalidation accurate?	Correct, spurious, and missed invalidations against full reruns; stale approvals; rebuild predictions	Replay of existing history
E1	Do generated evaluators discriminate?	Mutation score on known-bad and mutation-derived inputs, with and without controls	Audit of existing evaluators
E3	Does sequencing help?	Evaluation cost, time to decision, false rejection, articles consumed, on held-out changes	Replay of existing history
E4	Do physical results contradict checked claims?	Contradictions, control status, ledger representation (a case series)	One physical build
E5	Does it transfer and beat a baseline?	Authoring effort, coverage, stale approvals caught, builds avoided, candidates recovered	A project nopekit did not learn from
E6	Does accepted throughput saturate when $\Lambda > 1$?	Λ , σ , backlog, work-in-process, and lead time as G rises	An instrumented workflow
E7	Does raising B_F pay more when $\Lambda > 1$?	Change in A after cutting one evaluator’s latency, by regime	Two instrumented workflows
E0	Does Λ predict cross-domain leverage?	Rank correlation of the predicted Λ ordering with realized AI use	A cross-domain dataset

and divergence in lead time [36]; a plateau alone fits several other mechanisms.

E0 — Cross-domain prediction (pre-registered). We state now, before assembling any dataset, the ordering the framework predicts for five domains by median Λ , from lowest to highest: software, chip design, consumer hardware, laboratory workflows, buildings. We further predict that the rank correlation between this ordering and realized AI use [8, 26] remains significant after controlling for task exposure [19], corpus availability, and regulatory intensity. The design cannot establish causation: high-leverage domains have an incentive to invest in evaluators (reverse causation); conversational usage data over-represent tasks whose outputs are cheap to check, which is the predictor (endogeneity); and throttling pushes observed Λ toward 1 everywhere (attenuation). A null result would be weak evidence against the framework, and we will treat it as such. We report

the prediction so that it can be checked against us.

If falsification measures add little explanatory value after controlling for corpus, regulation, product availability, and adoption lag, the cross-domain thesis must be narrowed. If accepted throughput keeps rising with generation volume alone in workflows measured at $\Lambda > 1$, the conditional claim in the title is wrong. If sequencing or invalidation fails to outperform simpler baselines, those mechanisms should be revised rather than defended by analogy.

11 Discussion and limitations

Falsification capacity is an engineering resource whose value depends on more than evaluator count: fast checks add little assurance if they share failure modes, omit important claims, or cannot be connected to the decision being made. The framework distinguishes evidence from truth without eliminating the gap between them; known-bad controls expose evaluators that cannot fail but not evaluators that check the wrong property, so independent review and physical outcomes remain essential. The ledger bounds known unknowns only: a failure no one declared a claim about is not even a gap.

Evaluators encode expertise [22]: eliciting one takes practitioner time, tacit knowledge resists formalization, and the knowledge-acquisition bottleneck that limited expert systems applies with little modification. A library lowers authoring costs only when evaluators transfer across similar contexts. This is the framework’s principal adoption risk, and it has a precedent: design-rationale systems were repeatedly shown to be representationally adequate and practically unused, because capture overhead fell on the person who gained least from it [34]. Whether read-set tracing and control authoring avoid that fate is what E5’s authoring-effort measure is designed to decide.

Falsification is also not always the right response. Some decisions turn on preferences, social values, or institutional authority rather than feasibility, and some design problems have

no stopping rule. The framework keeps infeasible, dominated, preferred-against, and unresolved apart.

The broader scaling hypothesis is that generation rate is rising faster than independent falsification capacity, so that Λ rises across domains. The direction is not seriously in doubt: generation cost has fallen by orders of magnitude in three years, while the latency of a fabrication run, a clinical trial, or a thermal-cycling test has not moved. The open questions are the rate, the cross-domain variation, and whether the cycle-time race closes the gap. The practical implication is to measure Λ and invest in faster, cheaper, more independent evaluation wherever it unlocks value that more generation cannot.

12 Conclusion

Whether cheap possibilities become accepted outputs depends on the mechanisms available to reject, revise, or accept them. If the framework is right, the most consequential engineering investment of the next decade is not in larger generators but in the instruments that reject their output: faster simulation, autonomous laboratories, instrumented products, and shorter regulatory loops. That prediction is checkable, and Section 10 says how.

The important question is not simply how many possibilities a model can produce. It is how quickly, cheaply, and reliably a system can determine which of them deserve to proceed.

References

- [1] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967. doi:10.1145/1465482.1465560.
- [2] Y. Annpureddy, C. Liu, G. Fainekos, and S. Sankaranarayanan. S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In *TACAS, LNCS 6605*, pp. 254–257, 2011. doi:10.1007/978-3-642-19835-9_21.
- [3] ASME. *V&V 20-2009: Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer*. ASME, 2009.
- [4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The oracle problem in software testing: A survey. *IEEE Trans. Softw. Eng.*, 41(5):507–525, 2015. doi:10.1109/TSE.2014.2372785.
- [5] J. Becker, N. Rush, E. Barnes, and D. Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089, 2025.
- [6] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient detection of vacuity in temporal model checking. *Form. Methods Syst. Des.*, 18(2):141–163, 2001. doi:10.1023/A:1008779610539.
- [7] E. Berger, M. Usama, J. Mehlstäubl, B. Saske, and K. Paetzold-Byhain. Physics-in-the-loop: A hybrid agentic architecture for validated CAD engineering design. arXiv:2605.19717, 2026.
- [8] A. Bick, A. Blandin, and D. J. Deming. The rapid adoption of generative AI. NBER Working Paper 32966, 2024.
- [9] B. W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, 1984. doi:10.1109/MS.1984.233702.
- [10] E. Brynjolfsson, D. Li, and L. Raymond. Generative AI at work. *Q. J. Econ.*, 140(2):889–942, 2025. doi:10.1093/qje/qjae044.
- [11] B. Burger et al. A mobile robotic chemist. *Nature*, 583:237–241, 2020. doi:10.1038/s41586-020-2442-2.
- [12] C. Catalini, X. Hui, and J. Wu. Some simple economics of AGI. arXiv:2602.20946, 2026.
- [13] T. Y. Chen et al. Metamorphic testing: A review of challenges and opportunities. *ACM Comput. Surv.*, 51(1):1–27, 2018. doi:10.1145/3143561.
- [14] Google DeepMind. Millions of new materials discovered with deep learning. Blog, 2023. <https://deepmind.google/blog/millions-of-new-materials-discovered-with-deep-learning/>.
- [15] J. de Kleer. An assumption-based TMS. *Artif. Intell.*, 28(2):127–162, 1986. doi:10.1016/0004-3702(86)90080-9.
- [16] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi:10.1109/C-M.1978.218136.
- [17] D. E. Eckhardt and L. D. Lee. A theoretical basis for the analysis of multiversion software subject to coincident errors. *IEEE Trans. Softw. Eng.*, SE-11(12):1511–1517, 1985. doi:10.1109/TSE.1985.231895.
- [18] S. Elbaum, A. G. Malishevsky, and G. Rothermel. Test case prioritization: A family of empirical studies. *IEEE Trans. Softw. Eng.*, 28(2):159–182, 2002. doi:10.1109/32.988497.
- [19] T. Eloundou, S. Manning, P. Mishkin, and D. Rock. GPTs are GPTs: Labor market impact potential of LLMs. *Science*, 384(6702):1306–1308, 2024. doi:10.1126/science.adj0998.
- [20] S. D. Eppinger and T. R. Browning. *Design Structure Matrix Methods and Applications*. MIT Press, 2012.
- [21] G. E. Fainekos and G. J. Pappas. Robustness of temporal logic specifications for continuous-time signals. *Theor. Comput. Sci.*, 410(42):4262–4291, 2009. doi:10.1016/j.tcs.2009.06.021.
- [22] E. A. Feigenbaum. Knowledge engineering: The applied side of artificial intelligence. *Ann. N.Y. Acad. Sci.*, 426(1):91–107, 1984. doi:10.1111/j.1749-6632.1984.tb16513.x.
- [23] A. I. J. Forrester, A. Söbester, and A. J. Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. Wiley, 2008.
- [24] D. Guo et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi:10.1038/s41586-025-09422-z.
- [25] J. L. Gustafson. Reevaluating Amdahl’s law. *Commun. ACM*, 31(5):532–533, 1988. doi:10.1145/42411.42415.
- [26] K. Handa et al. Which economic tasks are performed with AI? Evidence from millions of Claude conversations. arXiv:2503.04761, 2025.
- [27] S. Harnad. The symbol grounding problem. *Physica D*, 42(1–3):335–346, 1990. doi:10.1016/0167-2789(90)90087-6.
- [28] J. M. Hellerstein and M. Stonebraker. Predicate migration: Optimizing queries with expensive predicates. In *SIGMOD*, pp. 267–276, 1993. doi:10.1145/170035.170078.
- [29] W. J. Hopp and M. L. Spearman. *Factory Physics*. 3rd ed., Waveland Press, 2011.
- [30] R. A. Howard. Information value theory. *IEEE Trans. Syst. Sci. Cybern.*, 2(1):22–26, 1966. doi:10.1109/TSSC.1966.300074.
- [31] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Trans. Softw. Eng.*, 37(5):649–678, 2011. doi:10.1109/TSE.2010.62.
- [32] J. F. C. Kingman. The single server queue in heavy traffic. *Math. Proc. Cambridge Philos. Soc.*, 57(4):902–904, 1961. doi:10.1017/S0305004100036094.
- [33] J. C. Knight and N. G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Trans. Softw. Eng.*, SE-12(1):96–109, 1986. doi:10.1109/TSE.1986.6312924.
- [34] J. Lee. Design rationale systems: Understanding the issues. *IEEE Expert*, 12(3):78–85, 1997. doi:10.1109/64.592267.
- [35] J. Leeman et al. Challenges in high-throughput inorganic materials prediction and autonomous synthesis. *PRX Energy*, 3:011002, 2024. doi:10.1103/PRXEnergy.3.011002.
- [36] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Oper. Res.*, 9(3):383–387, 1961. doi:10.1287/opre.9.3.383.
- [37] B. Littlewood and D. R. Miller. Conceptual modeling of coincident failures in multiversion software. *IEEE Trans. Softw. Eng.*, 15(12):1596–1614, 1989. doi:10.1109/32.58771.
- [38] A. Memon et al. Taming Google-scale continuous testing. In *ICSE-SEIP*, pp. 233–242, 2017. doi:10.1109/ICSE-SEIP.2017.16.
- [39] A. Merchant et al. Scaling deep learning for materials discovery. *Nature*, 624:80–85, 2023. doi:10.1038/s41586-023-06735-9.
- [40] A. Mokhov, N. Mitchell, and S. Peyton Jones. Build systems à la carte. *Proc. ACM Program. Lang.*, 2(ICFP):1–29, 2018. doi:10.1145/3236774.
- [41] S. A. Niederer, M. S. Sacks, M. Girolami, and K. Willcox. Scaling digital twins from the artisanal to the industrial. *Nat. Comput. Sci.*, 1(5):313–320, 2021. doi:10.1038/s43588-021-00072-5.
- [42] S. Noy and W. Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192, 2023. doi:10.1126/science.adh2586.
- [43] W. L. Oberkampf and C. J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010. doi:10.1017/CBO9780511760396.
- [44] B. Peherstorfer, K. Willcox, and M. Gunzburger. Survey of multifidelity methods in uncertainty propagation, inference, and optimization. *SIAM Review*, 60(3):550–591, 2018. doi:10.1137/16M1082469.
- [45] K. R. Popper. *The Logic of Scientific Discovery*. Hutchinson, 1959.
- [46] A. M. Ross and D. E. Hastings. The tradespace exploration paradigm. *INCOSE Int. Symp.*, 15(1):1706–1718, 2005. doi:10.1002/j.2334-5837.2005.tb00783.x.
- [47] G. Rothermel and M. J. Harrold. A safe, efficient regression test selection technique. *ACM Trans. Softw. Eng. Methodol.*, 6(2):173–210, 1997. doi:10.1145/248233.248262.
- [48] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *SIGMOD*, pp. 23–34, 1979. doi:10.1145/582095.582099.
- [49] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE*, 104(1):148–175, 2016. doi:10.1109/JPROC.2015.2494218.
- [50] D. K. Sobek, A. C. Ward, and J. K. Liker. Toyota’s principles of set-based concurrent engineering. *Sloan Manag. Rev.*, 40(2):67–83, 1999.
- [51] N. J. Szymanski et al. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624:86–91, 2023. doi:10.1038/s41586-023-06734-w.
- [52] T. Ünliüyurt. Sequential testing of complex systems: A review. *Discrete Appl. Math.*, 142(1–3):189–205, 2004. doi:10.1016/j.dam.2002.08.001.
- [53] J. Wei. Asymmetry of verification and verifier’s rule. Blog post (not peer reviewed), 2025. <https://www.jasonwei.net/blog/asymmetry-of-verification-and-verifiers-law>.